

San Diego Freeway Congestion Forecasting

A Graph Neural Network Approach to Joint Flow and Congestion Prediction on LargeST-SD

End-to-End Project Report

Engagement: Capstone project, traffic forecasting & operations analytics
Dataset: LargeST – San Diego subset, 716 sensors, 2019, 15-min resolution
Team: M1 (modeling / evaluation) • M2 (data / predictions) • M3 (dashboard)
M4 (chatbot) • M5 (integration / QA / deck)
Status date: September 19, 2026
Final review: September 21, 2026

Prepared as the complete technical and analytical record of the project.
For a short version, see the repository `README.md`.

Contents

1	Executive Summary	2
2	Business Context and Objective	2
2.1	Compute constraints, and why they matter to how this was built	3
3	Delivery Approach: A Gated Pipeline	3
4	Data Quality Audit (Gate o)	3
5	Constructing the Congestion Label (Stage L)	5
5.1	Cross-validation against LargeST	5
5.2	Critical occupancy and the three-class label	5
5.3	Validation	6
6	Incident Intelligence (Stage I)	6
7	Feature Engineering Program (Stage F)	7
8	Model Architecture	7
8.1	Core mechanics	8
8.2	Loss function	9
9	Training Protocol and Readiness Gates	9
10	Results	10
10.1	Headline performance	10
10.2	Full feature-ablation table	10
10.3	Graph ablation: how much does the graph structure matter?	10
10.4	Classification performance in context	10
10.5	Stratified diagnostics	12
11	Statistical Verification (Gate E)	12
12	Key Findings and Recommendations	14
13	Limitations	14
14	Solution Deployment: Dashboard and Chatbot	15
14.1	Map views and live KPIs	15
14.2	Agentic chatbot architecture	17
14.3	Bugs found and fixed	18
15	Reproducibility and Handover	19
16	Status and Roadmap	19

1 Executive Summary

The mandate was simple to state and hard to deliver: forecast freeway congestion in San Diego, 15 minutes to 3 hours ahead, at a level of accuracy that beats the client's own LSTM benchmark (mean absolute error, MAE, of 26.35 vehicles per 5 minutes) — and be able to prove, statistically, which of the engineering choices behind that number actually mattered.

FINDING

The shipped model (**R4**) reduces test-set forecasting error by **29.6%** against the client's LSTM baseline (18.56 vs. 26.35 average MAE across 12 horizons) and classifies live congestion state (free / heavy / congested) with a macro-F1 of **0.790** — within 5% of the best possible classification baseline available on this data (a train-fitted time-of-day profile, macro-F1 0.761), *and* beating it materially on the operationally important classes: heavy (F1 0.665 vs. 0.656) and congested (F1 0.730 vs. 0.667).

Three decisions separate this result from a routine model-fitting exercise, and each is traceable to a specific piece of evidence in this report:

1. **The congestion label itself had to be built, not assumed.** LargeST supplies flow only. A naïve flow-threshold label was tested and rejected outright — it flags 46.8% of Christmas Day as “congested” (the correct, occupancy-based answer: 0.03%) because it cannot distinguish low demand from a genuine jam. The label used throughout this report is instead fitted from PeMS occupancy, per sensor, on training data only, and independently validated against five separate criteria (Section 5).
2. **Every architectural and feature choice is backed by a statistical test, not a single-run delta.** A paired, day-block bootstrap (1,000 resamples across the 74 distinct calendar days in the test window) shows that two feature additions materially improved the model (utilization/time/static features, and holiday/school-calendar features) while two further additions (weather, incidents) did *not* move the aggregate metric outside of sampling noise — a negative result the team is reporting as such, rather than omitting (Section 11).
3. **The deliverable is a working system, not just a metric.** A local dashboard replays the full test period with a live map of predicted congestion, predicted-vs-actual flow, and incident overlays, backed by a chatbot that answers natural-language questions about predictions and model performance using function-calling tools grounded in the same data (Section 14).

The remainder of this report walks through the full pipeline end to end: data foundations, the gated delivery process used to de-risk an eight-day build on constrained hardware, the model itself, every result and its statistical backing, known limitations, and the deployed solution.

2 Business Context and Objective

San Diego's freeway network — 716 mainline sensors across I-5, I-8, I-15, I-805, and the state routes — produces a 15-minute flow reading at every sensor, every day. The client's own LSTM model, trained on this feed, achieves an average MAE of 26.35 vehicles per 5 minutes across a 12-step (3-hour) forecast horizon. The LargeST paper's own published GWNNet baseline, trained for roughly 80 epochs on a data-center GPU, achieves 17.74 — a meaningfully better number, but one that says nothing about *congestion state*, which is the quantity operations teams actually act on.

This project's objective was two-fold:

- **Beat the flow-forecasting baseline** using the same GWNet architecture the literature already validates, under real hardware constraints (a laptop GPU, not a data-center card).
- **Add a congestion classification capability** the LargeST benchmark does not provide at all, engineered and evaluated with the same rigor as the flow metric, and expose both through a working analytics tool.

2.1 Compute constraints, and why they matter to how this was built

The project was scoped for free-tier Google Colab (T4 GPU, session-limited). That plan failed in practice – the intended Colab-via-IDE remote-kernel route could not access the `google.colab` runtime – and every stage from 2026-09-13 onward, including all nine training runs, ran locally on an **Apple M2 Air with 8 GB of unified memory**, using Apple’s MPS backend rather than CUDA.

IMPLICATION

A consumer laptop is not a data-center GPU. Every run in this report used a micro-batch of 8 samples accumulated to an effective batch of 64, ran in fp32 (mixed precision measured only an 11% speedup on MPS – not worth the added complexity), and was capped at 16 epochs on a warmup → constant → cosine-decay learning-rate schedule, versus the paper’s much longer training budget (roughly 80 epochs on an A6000). A full run took approximately 4.8 hours. Every number in this report should be read against that budget: the gap to the paper’s own 17.74 MAE is compute, not methodology.

To make that constraint tractable, training runs were queued end-to-end, overnight, via a detached, resumable shell process that would pick up a checkpoint after any interruption (the harness running these sessions kills background watchers under memory pressure, so resumability was a design requirement, not a nicety) rather than a single unbroken job.

3 Delivery Approach: A Gated Pipeline

Rather than building the full system and evaluating it once at the end, the project was structured as a sequence of **gates** – checkpoints where a specific, falsifiable claim had to be verified before the next stage of work was authorized. This is a risk-management choice as much as a technical one: on a single laptop with a hard external deadline, discovering a flawed assumption late (say, in the final evaluation) would have been unrecoverable.

Every gate in Figure 1 produced a written, falsifiable result – not just “it ran.” The next four sections walk through what each gate actually found.

4 Data Quality Audit (Gate 0)

Before any model code was written, the raw LargeST data and the model’s own reproduction of the paper’s published baselines were audited end to end.

- The paper’s HL (historical-lag) baseline was **reproduced exactly**, confirming the local pipeline matches the published methodology bit-for-bit.
- The timestamp index is **wall-clock, not elapsed time** – the March 10 2:00–3:00am daylight-saving gap is all-zero in the data, confirmed rather than silently mishandled.
- Zero-flow readings are **outages, not genuine zero traffic**: 94% of zero-runs longer than a day coincide with clear sensor dropout signatures.

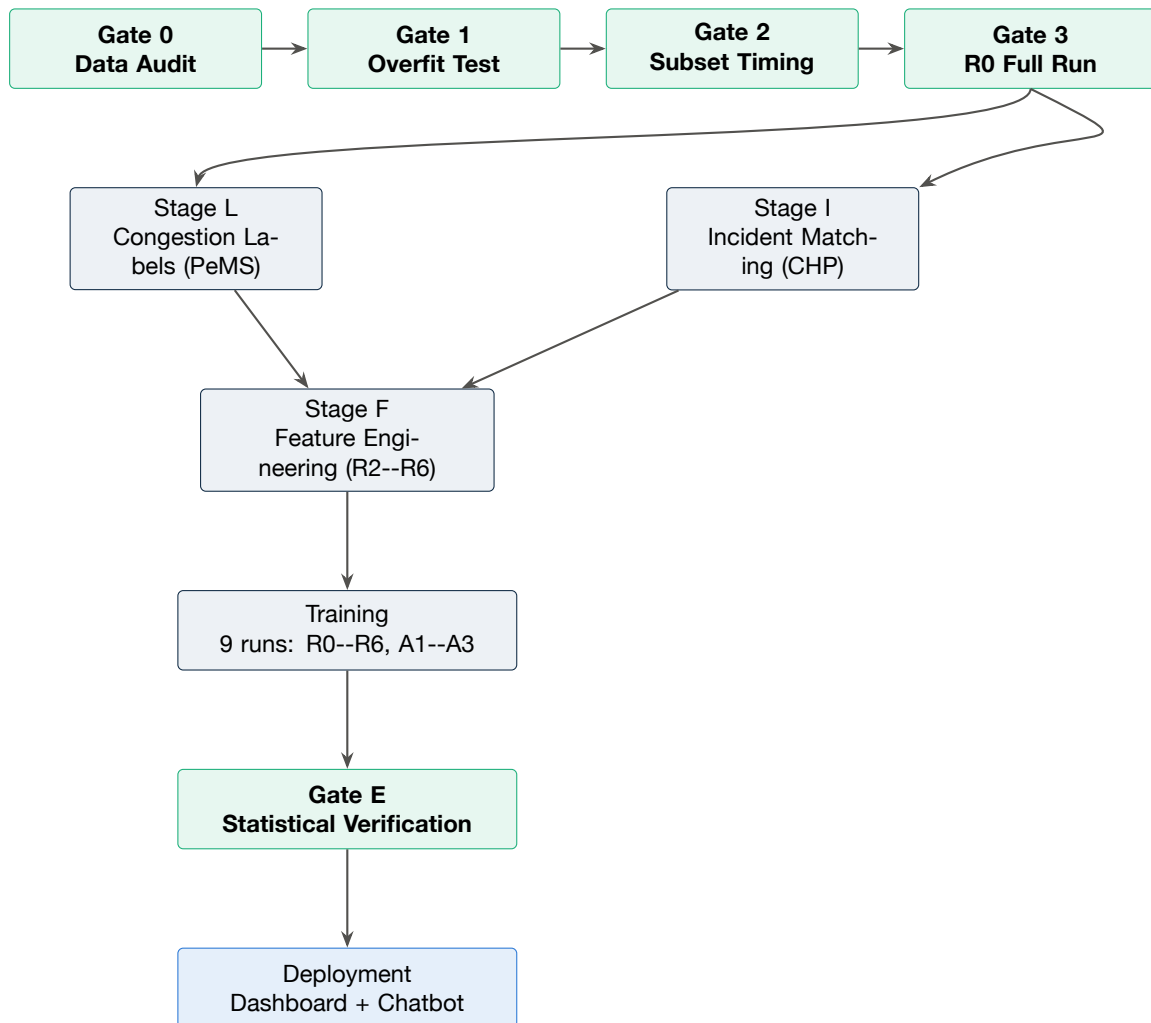


Figure 1: Gated development pipeline. Green boxes are hard checkpoints; each was reviewed before the next stage began. Gate 0b (a same-day follow-up to Gate 0) is folded into Gate 0 here for space.

- A hard **999 flow ceiling** appears on only 37 cells, all on 6–8 lane sensors – later independently confirmed against PeMS’s own 5-minute archive to be genuine field saturation, not a pipeline artifact.
- **5,049 cells imply a physically impossible rate** (over 3,200 vehicles per lane per hour), flagging lane-count metadata as suspect for a subset of sensors. On inspection, only one sensor (ID 1108333, metadata says 2 lanes, observed flow implies roughly 5) warranted a manual correction – a plausibility fix, not a full re-audit of all 716 sensors’ metadata.
- A simple **time-of-day profile baseline** already ties the LSTM benchmark at a 2-hour horizon and beats it at 3 hours (test MAE 29.8 vs. val 21.9) – an early signal that the LSTM bar, while a fair benchmark, is not an especially high one at longer horizons.

FINDING

A same-day follow-up (“Gate ob”) attributed roughly **75% of the validation-to-test MAE gap** to two calendar windows alone – Thanksgiving week and December 21–31 – with a Christmas Day profile-baseline MAE of 127 against a median-day MAE of 23. This single finding motivated the holiday/school-calendar feature set built later (Section 7, row R4) and is the reason the test split’s difficulty is not uniform across the quarter it covers.

5 Constructing the Congestion Label (Stage L)

LargeST’s only published quantity is flow. Congestion state – the thing operations teams actually respond to – does not exist in the dataset and had to be engineered from a different data source: raw PeMS District 11 occupancy readings, obtained independently and cross-validated against LargeST before being trusted.

5.1 Cross-validation against LargeST

Before using PeMS data for anything, its flow numbers were checked against LargeST’s own: 15-minute flow matches in 99.98% of cells (matched independently by sensor ID and wall-clock time), and hourly aggregates correlate at 0.99998. Two asymmetries were found and are documented rather than smoothed over: LargeST silently drops 5-minute flow readings above 999 (quietly biasing those specific windows downward), and PeMS has genuine, non-imputed readings for 17% of the sensor-hours LargeST is missing entirely (86 sensors) – meaning PeMS is a strict informational gain, not a lossier substitute.

5.2 Critical occupancy and the three-class label

For each sensor, a **critical occupancy** o_c is fit on training-period data only: it is the occupancy value at which the upper envelope (95th percentile) of the flow–occupancy relationship peaks and then declines – the classical signature of a queue forming downstream of that detector. Formally, with $q(o)$ the smoothed p95 flow envelope as a function of binned occupancy o :

$$o_c = \arg \max_o q(o), \quad \text{subject to } \min_{o' > o_c} q(o') < 0.95 \cdot q(o_c) \quad (1)$$

Sensors with no observed decline after their flow peak (no congested branch was ever seen in training data) fall back to the network median o_c ; this affected 392 of 716 sensors, versus 278 with a directly fitted value (46 had too little data to fit at all). Given o_c , the hourly label is:

$$\text{class}(t) = \begin{cases} \text{congested} & \text{occupancy}(t)/o_c \geq 1.0 \\ \text{heavy} & 0.7 \leq \text{occupancy}(t)/o_c < 1.0 \\ \text{free} & \text{occupancy}(t)/o_c < 0.7 \end{cases} \quad (2)$$

and is repeated across the four 15-minute steps inside each labeled hour.

5.3 Validation

The label was checked five separate ways before being trusted as the training target:

- **Stability across splits:** class shares are consistent train/val/test (roughly 85–88% free, 7–9% heavy, 5–6% congested).
- **Face validity:** congestion peaks at 07:00 and 16:00 on weekdays; the ten highest-congestion sensors by weekday-peak share are recognizable real bottlenecks (SR-78 at Nordahl, 78% of weekday peak hours; SR-163 at Robinson Ave, 65%; I-5 at Poinsettia Ln, 60–61%).
- **Resolution robustness:** an independently built, genuinely 15-minute label for one week (Nov 4–10) agrees with the hourly-repeated label 95.7% of the time (congested recall 0.90, precision 0.85) – the simplification loses some precision but is directionally sound.
- **Spatial coherence:** congestion propagates to neighboring sensors at $12.9\times$ the rate of a random sensor pair (versus $3.6\times$ for a distance-matched control) – it behaves like a real traffic queue, not sensor noise.

FINDING

A naïve alternative – labeling “congested” from flow alone, without occupancy – was tested and rejected. It achieves precision 0.07 and recall 0.03 against the occupancy-based label, and, most tellingly, flags **46.8% of Christmas Day** as congested (the occupancy-based answer: **0.03%**). Flow alone cannot distinguish a genuine traffic jam from a holiday with almost no cars on the road. This is the single most important methodological decision behind the congestion-classification half of this project.

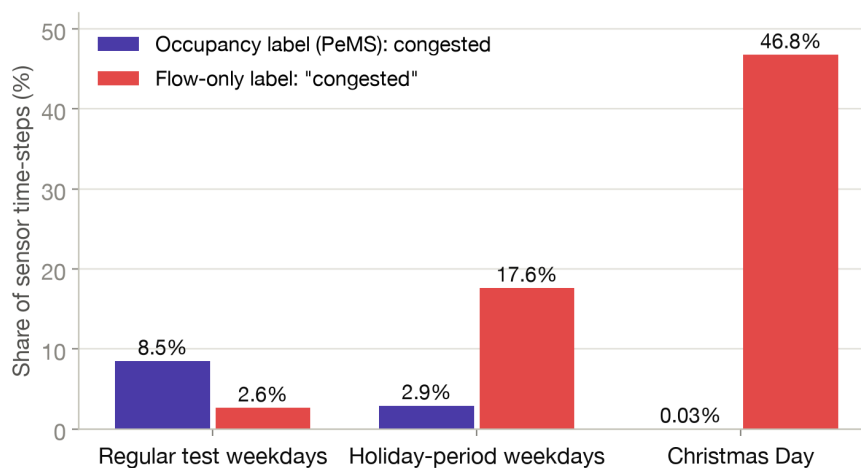


Figure 2: Occupancy-based label vs. a flow-only proxy label, share of sensor time-steps flagged “congested,” by calendar period. The flow-only label inverts on holidays.

6 Incident Intelligence (Stage I)

49,198 California Highway Patrol incident records for District 11 were classified, parsed for lanes-blocked where the free-text record allowed it (possible for 9.7% of records; 15.5% of collisions specifically), and matched to the nearest sensor – 92.6% matched successfully.

An event-study around matched incident start times shows congestion at the nearest upstream sensor rising by roughly **+14 percentage points** at the incident hour itself, with a smaller **+6-point pre-rise** one hour *before* the recorded start (congestion likely contributes to the incident, not only follows it) and a decay back to baseline over about three hours. In the test split, sensor-hours matched to an active incident show a 14.8% congested share versus 5.2% for non-incident cells – a real signal, though incident-affected cells are only about 3% of the test set (a point returned to in Section 11 and Section 13).

7 Feature Engineering Program (Stage F)

Six model configurations were trained on a strictly **cumulative** feature schedule, so that every row’s improvement (or lack of one) over the previous row isolates exactly one engineering decision.

Run	Adds on top of the previous row	Channels
R0	Base GWNNet, LargeST’s own 3 channels (flow, time-of-day, day-of-week); flow-only, single-task	3
R1	Same inputs as R0, plus the multitask congestion head	3
R2	Missing-data mask and causal fill for missing input flow	4
R3	Utilization, flow-per-lane, cyclical time features, static per-sensor attributes	33
R4	Holidays, day-before/after, holiday-period flag, school-in-session (SDUSD calendar)	38
R5	Weather (5 San Diego ASOS stations, inverse-distance-weighted, causal)	46
R6	CHP incident features	50

Table 1: Cumulative feature schedule, R0 through R6. Three further graph-ablation rows (A1–A3, Section 10.3) reuse R0’s input set to isolate the model’s graph structure instead.

Missing-input fill (from R2 onward) was chosen empirically, not by convention: four candidate fill strategies were scored against 1% of observed cells artificially hidden in the validation split. “Same weekday, last 4 weeks” (median fill) scored best (MAE 16.9), beating a train-only profile (21.9), a simple previous-7-days median (34.6), and zero-fill – effectively what LargeST does by default (255.0, i.e. far worse). Missing input cells are 2.28% of the total.

Weather (R5) draws on NOAA/Iowa State ASOS routine METARs from five stations (SAN/MYF/SEE/CRQ/), inverse-distance-weighted onto each sensor and strictly causal – no future weather can leak into a training window. Rain affects an uneven 7.3% of training steps, 0.6% of validation steps, and 8.4% of test steps (roughly 147 measurable hours) – relevant context for how the weather result in Section 11 should be read.

8 Model Architecture

The base model is **Graph WaveNet (GWNNet)**, the same architecture the LargeST paper benchmarks (311,164 parameters, matching the paper’s reported count), extended here with a **multi-task congestion classification head** sharing the same spatio-temporal backbone.

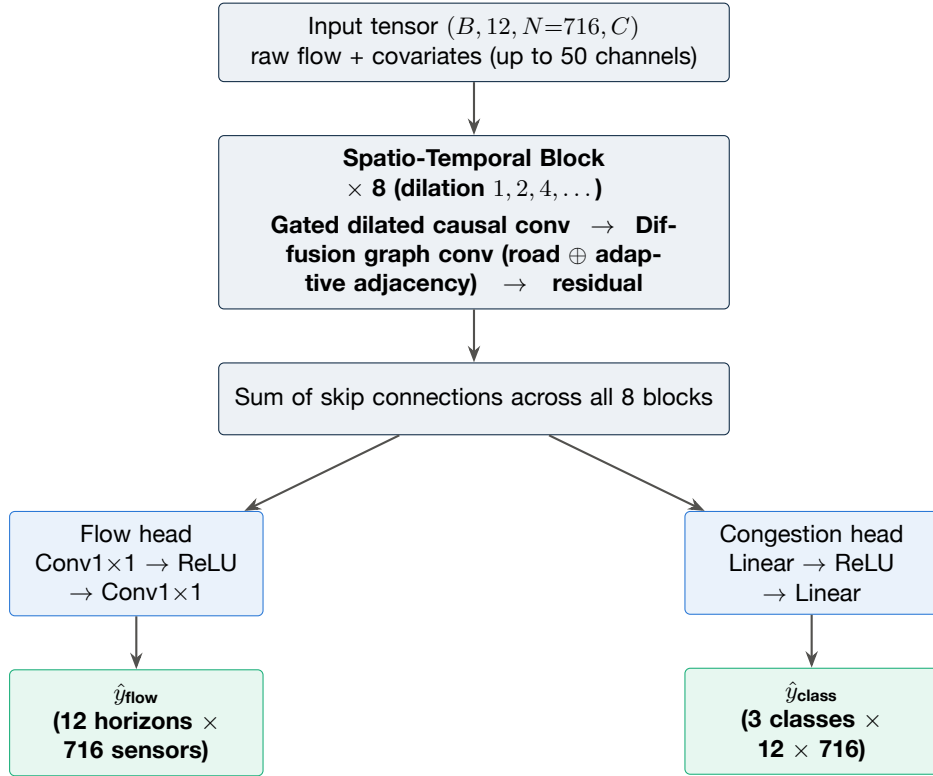


Figure 3: GWNet backbone with the multitask congestion head. The classification head branches off the same shared skip-connection representation the flow head reads from – both tasks see identical spatio-temporal features, differing only in their final projection.

8.1 Core mechanics

Each spatio-temporal block combines a **gated dilated causal convolution** (the WaveNet half) with a **diffusion graph convolution** (the spatial half):

$$h = \tanh(W_f * x) \odot \sigma(W_g * x) \quad (3)$$

$$Z = \sum_{k=0}^K \left(P_f^k h \Theta_{f,k} + P_b^k h \Theta_{b,k} \right) \quad (4)$$

where $*$ is a dilated causal convolution along the time axis, $\odot$ is element-wise product, and P_f, P_b are the forward and backward random-walk transition matrices derived from an adjacency A . GWNet’s key architectural contribution is that A is not fixed to the physical road network alone – it also learns a data-driven **adaptive adjacency**:

$$A_{\text{adp}} = \text{softmax}(\text{ReLU}(E_1 E_2^\top)) \quad (5)$$

from two learnable node-embedding matrices $E_1, E_2 \in \mathbb{R}^{716 \times 10}$ (14,320 parameters total) – letting the model discover statistical relationships between sensors the physical road graph does not encode (for instance, correlated but non-adjacent corridors). Section 10.3 quantifies exactly how much each adjacency source contributes.

IMPLICATION

One architectural quirk was identified and is reported for completeness, not as a defect: the base GWNet’s final graph-convolution/batch-norm layer (7,264 of the 311,164 parameters) never receives gradient, because the network’s output is read from the skip connections *before*

that final layer. This does not affect correctness – the paper’s own published parameter count includes these unused parameters too – but it is worth knowing before reading too much into a raw parameter count as a proxy for effective model capacity.

8.2 Loss function

The flow task uses LargeST’s own **masked MAE** – cells where the true label is exactly zero (a missing-data sentinel, not genuine zero flow; see Section 4) are excluded from the loss and from every reported metric:

$$\mathcal{L}_{\text{flow}} = \frac{\sum_i \mathbb{I}[y_i \neq 0] |\hat{y}_i - y_i|}{\sum_i \mathbb{I}[y_i \neq 0]} \quad (6)$$

The congestion task uses a class-weighted cross-entropy, with weights set to the inverse square root of each class’s training-set frequency ($w_c = 1/\sqrt{f_c}$) to counteract the roughly 17:1 imbalance between the free and congested classes. The two objectives are combined as

$$\mathcal{L} = \mathcal{L}_{\text{flow}} + \lambda \mathcal{L}_{\text{class}}, \quad \lambda = 15 \quad (7)$$

λ was fixed at 15 across every multitask run (R1 through R6) rather than tuned per row, so that the feature-ablation comparisons in Section 10 isolate the effect of the input features alone.

9 Training Protocol and Readiness Gates

Three gates preceded any full training run, each designed to catch a specific class of failure cheaply before committing laptop-hours to it.

Gate 1 -- Overfit Test.

A full 100-window memorization run is computationally infeasible for this model (100 windows $\times$ 12 horizons $\times$ 716 sensors implies 859,000 targets against roughly 304,000 trainable parameters). The actual test was a 4-window, full-batch memorization run, which must drive training loss below 10% of its starting value and keep falling – passed on the M2 (loss fell from 152.4 to 4.6) – backed by the 100-window run as a supplementary sanity curve, not the primary pass/fail criterion.

Gate 2 -- Subset Timing.

Five epochs on a 60-day data subset, intended to validate throughput and surface obvious training-loop bugs cheaply. It failed one of its stated criteria (beating the time-of-day profile baseline at horizons 1–3) – a result later shown by Gate 3 to be simple undertraining at 5 epochs, not a flaw in the setup.

Gate 3 -- First 25% of R0.

Three epochs on the *full* dataset. Validation-subset MAE fell from 43.4 to 26.5, beating both the historical-lag and profile baselines at horizons 1–3 (the profile baseline remained ahead at horizon ≥ 4 , as expected at only 3 epochs). This gate passed and fixed the final training recipe used for every subsequent run: **16 epochs**, seed 2023, a warmup $\rightarrow$ constant $\rightarrow$ cosine-decay learning-rate schedule (the decay covering the final 30% of training).

Nine runs followed this exact recipe: R0 through R6 (the feature ablation, Section 7) and A1 through A3 (the graph ablation, Section 10.3). They were executed in a fixed dependency order – R2 $\rightarrow$ A2 $\rightarrow$ R1 $\rightarrow$ R3 $\rightarrow$ R4 $\rightarrow$ R5 $\rightarrow$ R6, with A1/A3 run earlier – via a detached, resumable queue script, completing over roughly 65 hours of wall-clock time between 2026-09-15 and 2026-09-18.

10 Results

All figures in this section are **test-set** averages across all 12 forecast horizons (15 minutes to 3 hours ahead) unless stated otherwise.

10.1 Headline performance

Model	avg MAE	avg RMSE	avg MAPE
HL (historical lag, paper)	60.79	–	–
Time-of-day profile (ours)	29.8	–	–
LSTM (client baseline)	26.35	–	–
GWNet (paper, flow-only, ~80 epochs)	17.74	–	–
R4 (this project)	18.56	30.97	12.25%

Table 2: Headline comparison. R4 is 29.6% below the client’s LSTM baseline and within 4.6% of the paper’s own flow-only GWNet trained under a far larger compute budget.

10.2 Full feature-ablation table

Run	avg MAE	avg RMSE	avg MAPE	macro-F1	F1: free / heavy / congested
R0	20.35	33.48	13.40%	–	–
R1	20.41	33.70	13.65%	0.768	–
R2	20.39	33.63	13.81%	0.769	–
R3	18.90	31.60	12.63%	0.780	–
R4	18.56	30.97	12.25%	0.790	0.975 / 0.665 / 0.730
R5	18.64	31.03	12.27%	0.787	–
R6	18.70	31.22	12.26%	0.787	–

Table 3: Cumulative feature ablation, test set. R4 is both the lowest-error and highest-F1 row.

10.3 Graph ablation: how much does the graph structure matter?

Four configurations, all trained on R0’s input set, isolate the contribution of GWNet’s two adjacency sources (Section 8.1).

FINDING

Adaptive adjacency alone recovers roughly **80%** of the full model’s improvement over no-graph (21.84 vs. 27.92 vs. 20.35); road adjacency alone recovers roughly **66%** (22.90). Both together are better than either alone – confirmed as a statistically real gap in Section 11 – meaning the model is not simply re-deriving the road network from data: it is learning something the physical graph does not encode.

10.4 Classification performance in context

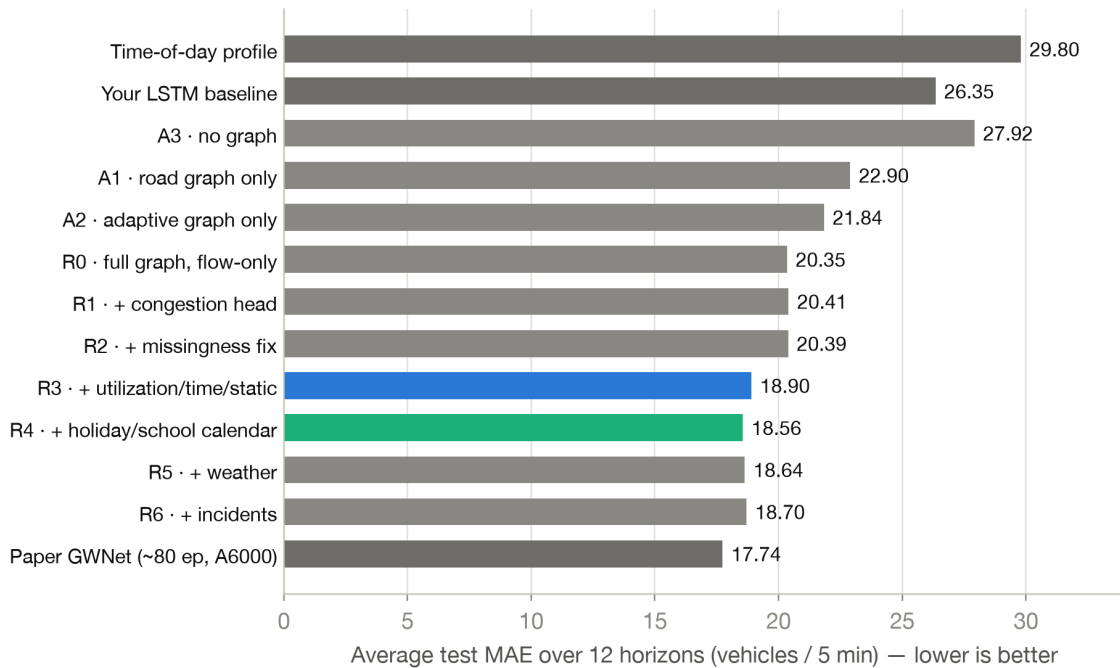


Figure 4: Average test MAE, every run and baseline. R3 and R4 (highlighted) are the two rows a statistical test confirms as real improvements – see Section 11.

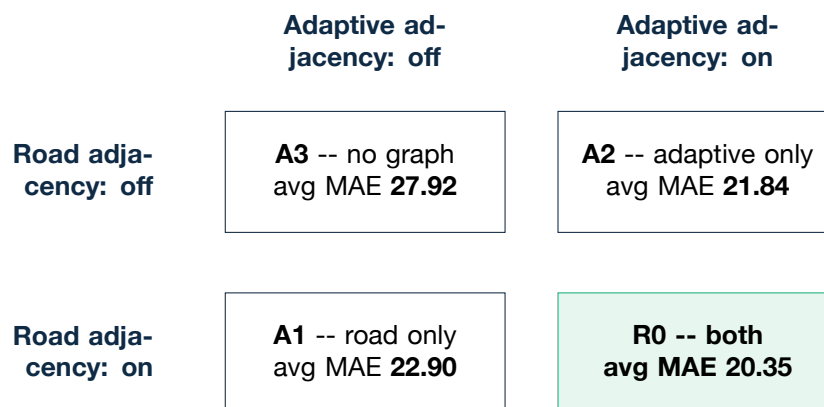


Figure 5: Graph ablation, 2×2. The two adjacency sources are complementary, not redundant – using both beats either alone.

Baseline / model	macro-F1	free	heavy	congested
		(recall, except R4 which reports F1)		
Persistence (hold last known class)	0.656	0.951	0.496	0.522
Profile-class (train-fitted majority per sensor/slot/day-type)	0.761	0.969	0.656	0.667
R4	0.790	0.975	0.665	0.730

Table 4: R4 against the two strongest available classification baselines. The profile-class baseline is a strong bar precisely because congestion is highly time-of-day-dependent – R4 still improves meaningfully on the operationally critical heavy/congested classes.

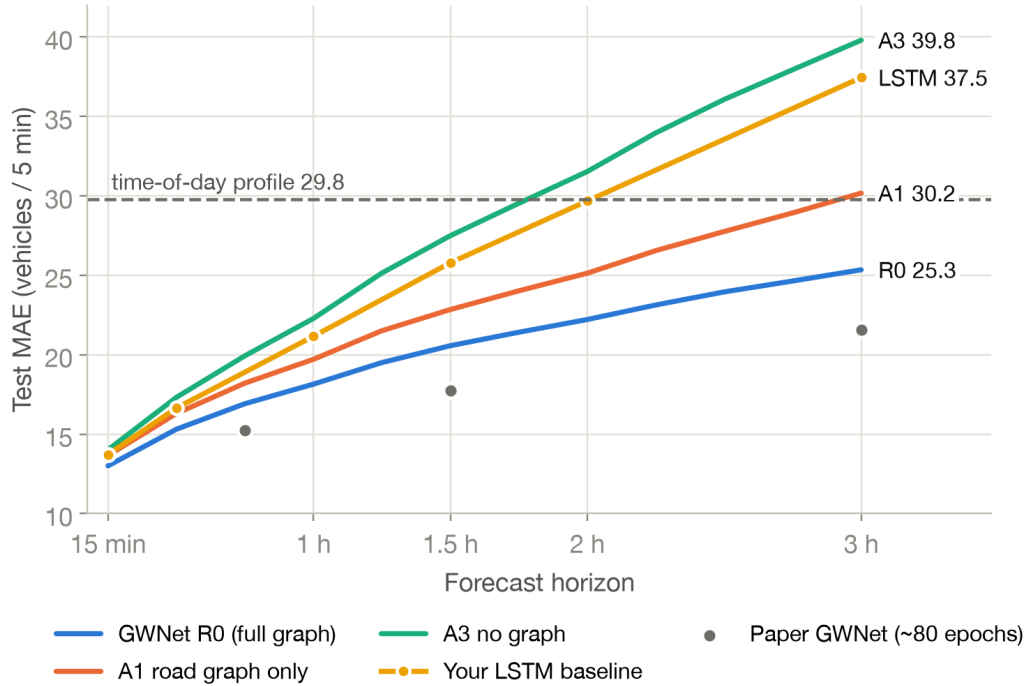


Figure 6: Test MAE by forecast horizon, graph-ablation rows against both baselines. The gap between configurations widens, not narrows, at longer horizons.

10.5 Stratified diagnostics

Every model was also evaluated on eight overlapping subsets of the test set – rain vs. dry, incident-affected vs. not, peak vs. off-peak, holiday-period vs. not, and each true congestion class – to check whether the aggregate number hides an important asymmetry. It does.

Run	overall	rain	dry	incident	non-incid.	peak	off-peak	holiday	non-hol.	free	heavy	congested
R0	20.35	28.17	19.64	30.09	20.03	32.07	17.27	24.74	19.74	17.46	30.09	44.33
A1	22.90	29.87	22.27	32.69	22.59	36.02	19.46	26.62	22.38	19.94	35.18	49.46
A2	21.84	29.83	21.12	31.83	21.52	35.24	18.32	26.06	21.25	18.70	32.75	47.88
A3	27.92	32.42	27.51	38.20	27.59	45.57	23.28	28.76	27.80	24.25	46.58	61.97
R1	20.41	28.37	19.68	30.21	20.09	32.14	17.33	25.39	19.71	17.60	29.99	42.78
R2	20.39	29.23	19.59	30.02	20.08	32.69	17.16	25.88	19.62	17.68	29.37	42.50
R3	18.90	26.66	18.20	28.76	18.59	29.81	16.04	24.68	18.10	16.04	27.45	40.87
R4	18.56	26.15	17.87	28.40	18.24	29.21	15.76	23.83	17.82	15.65	27.40	40.60
R5	18.64	25.79	17.99	28.42	18.32	29.30	15.84	23.87	17.91	15.76	27.11	40.29
R6	18.70	26.10	18.03	28.53	18.39	29.49	15.87	24.02	17.96	15.81	27.29	40.52

Table 5: Stratified test flow-MAE, every run $\times$ every stratum. Mask coverage: rain 8.30%, incident 3.07%, peak 20.76%, holiday-period 12.32%, true free/heavy/congested 74.58% / 6.21% / 4.67% of test sensor-timesteps.

11 Statistical Verification (Gate E)

A row-to-row improvement in Section 10 could, in principle, be sampling noise rather than a real effect of the feature or architecture change behind it. Gate E answers this directly with a **paired, day-block bootstrap**: the 74 distinct calendar days the test set’s forecast origins fall on are resampled with replacement 1,000 times; for each resample, the MAE delta between two models

is recomputed from only the resampled days, giving an empirical distribution of the delta and a 95% confidence interval by the percentile method:

$$CI_{95\%} = [Q_{2.5}(\Delta^*), Q_{97.5}(\Delta^*)], \quad \Delta_b^* = \overline{MAE}_{to}(D_b) - \overline{MAE}_{from}(D_b) \quad (8)$$

where D_b is the b -th resample of test days (with replacement) and $\overline{MAE}(D_b)$ is that model's MAE recomputed over only the windows originating on days in D_b . Blocking by day (rather than treating every 15-minute window as an independent sample) accounts for the fact that forecast errors on nearby windows within the same day are correlated, not independent draws.

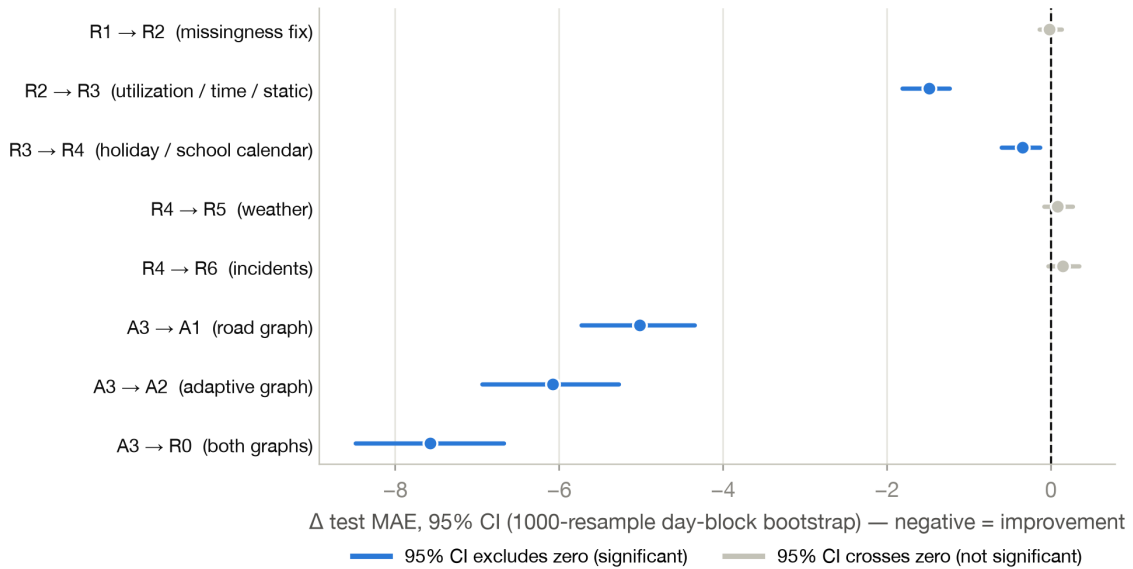


Figure 7: Bootstrap 95% confidence intervals on eight row-to-row deltas. A blue interval excludes zero (statistically real); a gray interval crosses it (not distinguishable from noise).

Comparison	ΔMAE	95% CI	Significant?
R1 → R2 (missingness fix)	-0.018	[-0.140, 0.135]	No
R2 → R3 (utilization / time / static)	-1.486	[-1.811, -1.233]	Yes
R3 → R4 (holiday / school calendar)	-0.346	[-0.604, -0.131]	Yes
R4 → R5 (weather)	+0.080	[-0.082, 0.264]	No
R4 → R6 (incidents)	+0.144	[-0.029, 0.343]	No
A3 → A1 (road graph)	-5.020	[-5.732, -4.345]	Yes
A3 → A2 (adaptive graph)	-6.079	[-6.939, -5.274]	Yes
A3 → R0 (both graphs)	-7.574	[-8.488, -6.680]	Yes

Table 6: Gate E bootstrap results, 1,000 resamples over 74 test-origin days.

FINDING

R4 is the statistically best model, not merely the lowest-MAE one. The R2→R3 and R3→R4 improvements are real by this test; R4→R5 and R4→R6 are not. But the aggregate result for R5 and R6 hides an asymmetry visible directly in Table 5: inside the *rain* stratum specifically, R5 improves on R4 (26.15 → 25.79 MAE) – a genuine, if minor, effect masked in aggregate only because rain covers just 8.3% of test cells. Inside the *incident* stratum specifically, R6 does *not* improve on R4 (28.40 → 28.53, worse) – unlike weather, the incident

feature shows no localized benefit even where incidents are active, which is reported here as a genuine negative result rather than quietly dropped.

IMPLICATION

The decision taken at this gate was to **ship R4** as the model behind the dashboard, the chat-bot, and the final review deck – not R6, despite R6 having strictly more engineered features. More features are not automatically better; the bootstrap is what separates a genuine improvement from noise dressed up as one, and that discipline is what this project is recommending as a general practice, independent of this specific dataset.

12 Key Findings and Recommendations

1. **Ship R4, not R6.** Every additional engineered feature increases system complexity and maintenance burden. R6 has the most features and is not the best model by a statistical test – a reminder to gate feature additions on evidence, not on effort invested.
2. **Congestion labels built from occupancy, not flow, are a prerequisite for any congestion-classification product on this kind of sensor network.** The flow-only proxy's 46.8%-of-Christmas-Day failure mode would have silently corrupted any downstream alerting or reporting built on it.
3. **Both graph sources earn their place.** The learned adaptive adjacency is not simply rediscovering the physical road network – it recovers more of the total graph benefit alone (80%) than the road graph does alone (66%), and using both is better than either. For a network expansion or a new region without a clean road-adjacency file, this suggests the adaptive mechanism alone is already most of the value.
4. **Weather is a real, narrow-scope win; incidents currently are not.** Both are reasonable hypotheses to test on any traffic network; this project's evidence is that only one of the two paid off here, and by how much (Section 11) – a result that should inform, not discourage, testing the same features on a larger or wetter dataset where the strata would carry more weight.
5. **Holiday-period behavior is disproportionately represented in this test window** and disproportionately hard to predict (Section 4, Section 13) – a structural property of a Q4-only test split that any production deployment on a similar calendar footprint should plan around, e.g. with holiday-specific model variants or wider training data coverage.

13 Limitations

LIMITATION

Congestion is measured, not observed directly. The label is derived from PeMS occupancy – itself an inferred signal from per-lane loop-detector on-time, not a direct queue measurement. Thresholds are fitted per sensor and validated five ways (Section 5), but remain a modeling choice.

LIMITATION

PeMS occupancy is itself partly imputed. Roughly 10% of underlying sensor-hours are below 50% observed and were excluded from *label fitting*; the *flow* series those hours feed via LargeST was not separately filtered.

LIMITATION

Test-only holidays. Thanksgiving and December 21–31 appear only in the test split, never in training or validation. The model must generalize holiday behavior from smaller training-set analogues (New Year's, Memorial Day, July 4th) – R4's calendar-feature gain is real on this split, but the largest test-period anomalies are exactly the days with the least similar training exposure.

LIMITATION

Single seed, fixed epoch budget. Every run used one seed (2023) and 16 epochs, against the paper's roughly 80-epoch, data-center training budget. The bootstrap in Section 11 quantifies *test-sampling* uncertainty only, not *training-seed* uncertainty, which was not measured given the available compute.

LIMITATION

Weather and incident coverage are limited by what actually occurred in the test period – about 147 hours of measurable rain, about 3% incident-affected cells. Real effects, on a comparatively small slice of the data; the R5/R6 stratified findings in Section 11 should be read as suggestive, not as tightly bounded estimates.

LIMITATION

One hand-corrected sensor. Lane count for sensor 1108333 was corrected manually (metadata said 2 lanes; observed flow implies roughly 5) – a targeted plausibility fix, not a systematic re-audit of all 716 sensors' metadata.

14 Solution Deployment: Dashboard and Chatbot

The final deliverable pairs R4 with a working analytics tool, built to run entirely locally – a deliberate choice after weighing a hosted deployment's free-tier resource limits against the timeline to the final review.

14.1 Map views and live KPIs

Four synchronized map views, all driven by the same sidebar date / time / horizon controls: predicted congestion state, predicted-vs-actual flow (color = signed % error), an incident overlay, and an error-magnitude view for model-quality diagnosis separate from the raw comparison – plus a sensor drill-down showing actual-vs-predicted flow across a chosen day at multiple horizons. A **KPI header row** (network average predicted flow, % congested/heavy, active CHP incidents, this window's MAE against the model's overall average) recomputes live off whatever the sidebar is set to; an **ops watchlist** ranks sensors congested-first for the current slot; an **auto-play** control steps the time slider forward automatically; and a **model-performance view** brings the ablation chart, the Gate E bootstrap forest plot, and the Section 10–11 findings directly into the app as FINDING/LIMITATION callouts.

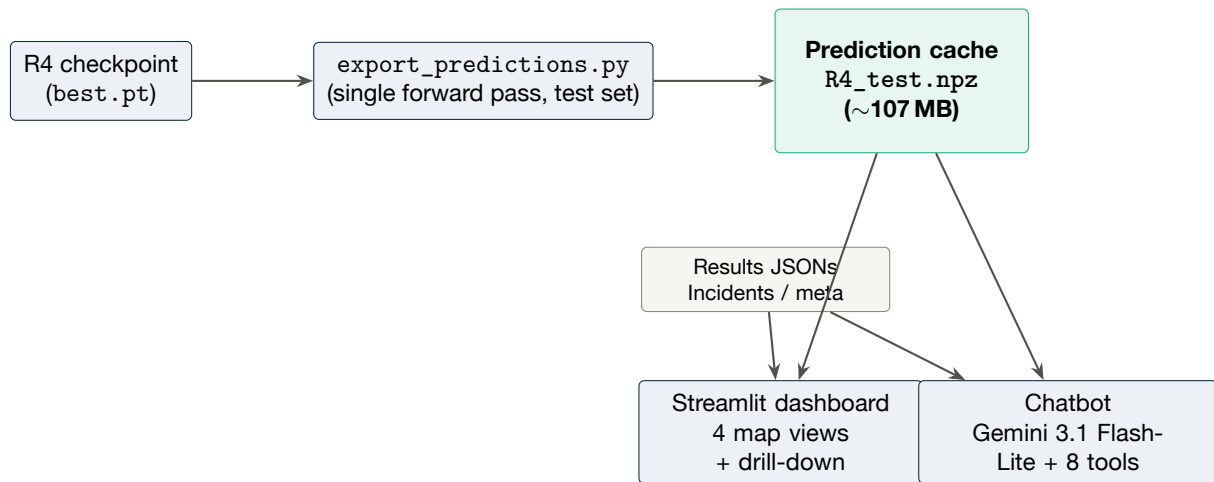


Figure 8: Dashboard and chatbot data flow. Neither surface requires PyTorch at runtime – only the prediction cache and small JSON/metadata files, which keeps the running application lightweight.

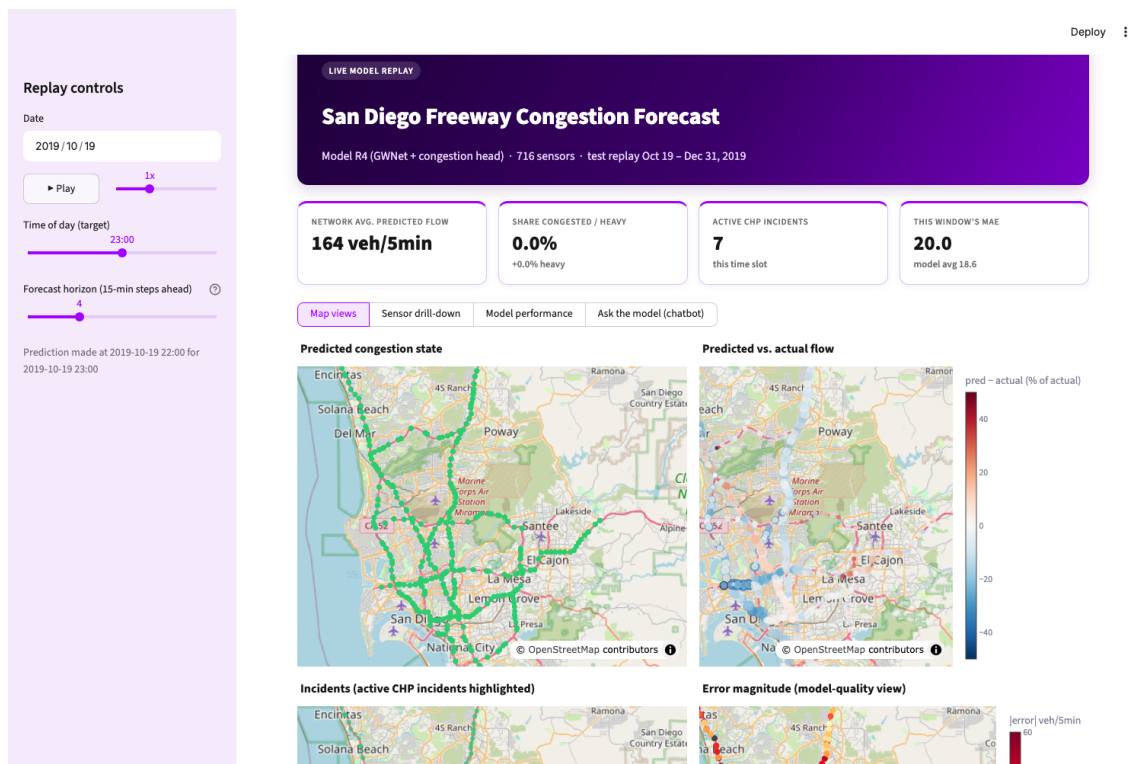


Figure 9: The dashboard's Map views, showing the hero banner, live KPI row, and the four synchronized map blocks – Accenture purple (#A100FF) governs chrome only, never the traffic-domain semantic colors (free/heavy/congested) in the maps themselves.

14.2 Agentic chatbot architecture

The chatbot was originally scoped against the Claude API and, on request, rebuilt against **Google Gemini's free tier** – then deliberately upgraded from basic tool-calling into something that reads as genuinely agentic: visible reasoning, autonomous multi-step investigation, an ability to *act* on the dashboard rather than only describe it, and a self-check before presenting an answer.

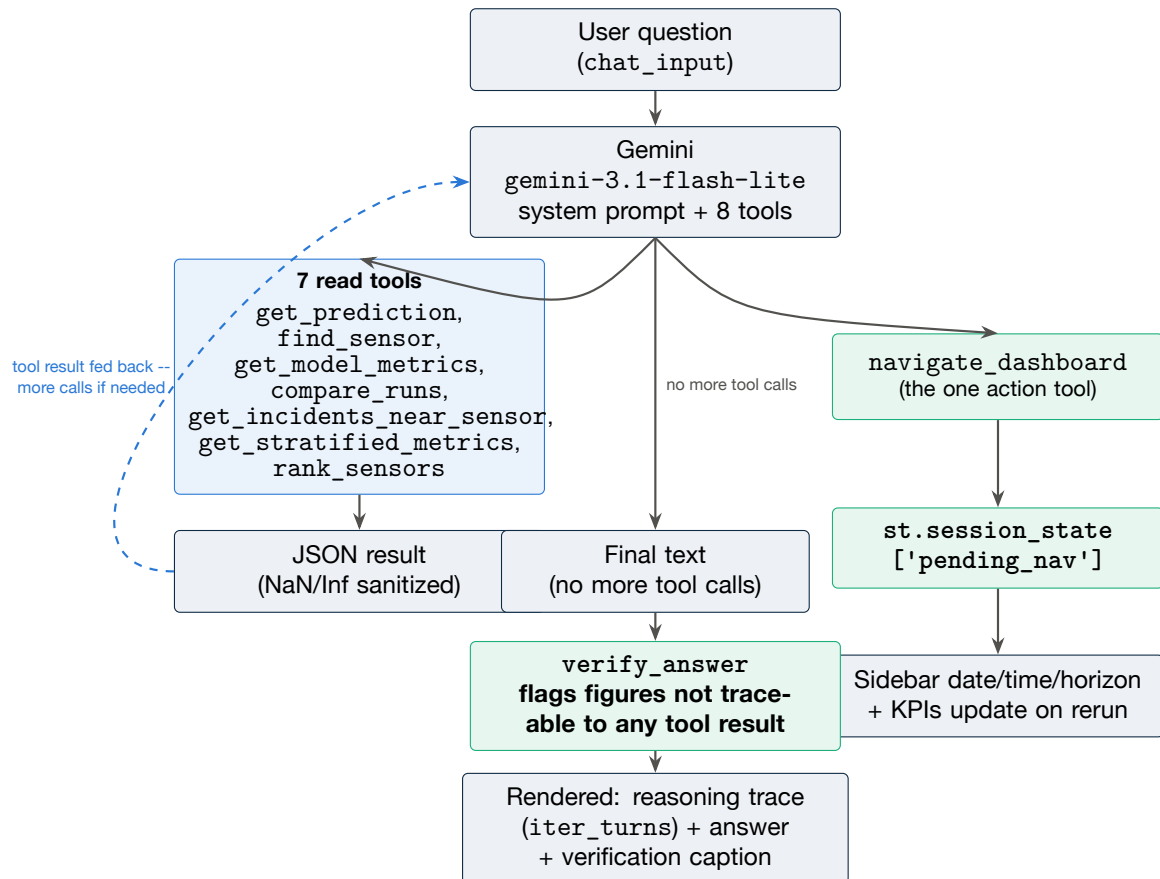


Figure 10: The agentic tool-use loop behind the chatbot. `navigate_dashboard` is the only tool with a side effect beyond returning data – it is what lets the agent act on the dashboard rather than only describe it. The dashed steel arrow is the loop: up to 8 rounds of tool calls can chain before the model produces a final answer.

- **Eight tools, not five** – two additions specifically enable the autonomous multi-step investigation in Figure 10: `get_stratified_metrics` exposes the rain/incident/peak/holiday breakdown computed for Gate E (Section 11) but never reachable via chat before; `rank_sensors` ranks all 716 sensors at any arbitrary time by congestion, error, or flow, not just the dashboard's currently-displayed slice.
- **One action tool.** `navigate_dashboard` validates the requested time against the test window, then writes `st.session_state['pending_nav']`, which the dashboard resolves into the sidebar's date/time/horizon on the next render – the one tool whose dispatch function has a side effect beyond returning data.
- **A visible reasoning trace.** `chatbot.iter_turns()` derives a per-turn `{steps, text}` grouping purely from the existing conversation history at render time (no separate bookkeeping structure, to avoid an entire class of state-desync bug), rendered as an expandable panel listing every tool call, its arguments, and its result before the final answer.

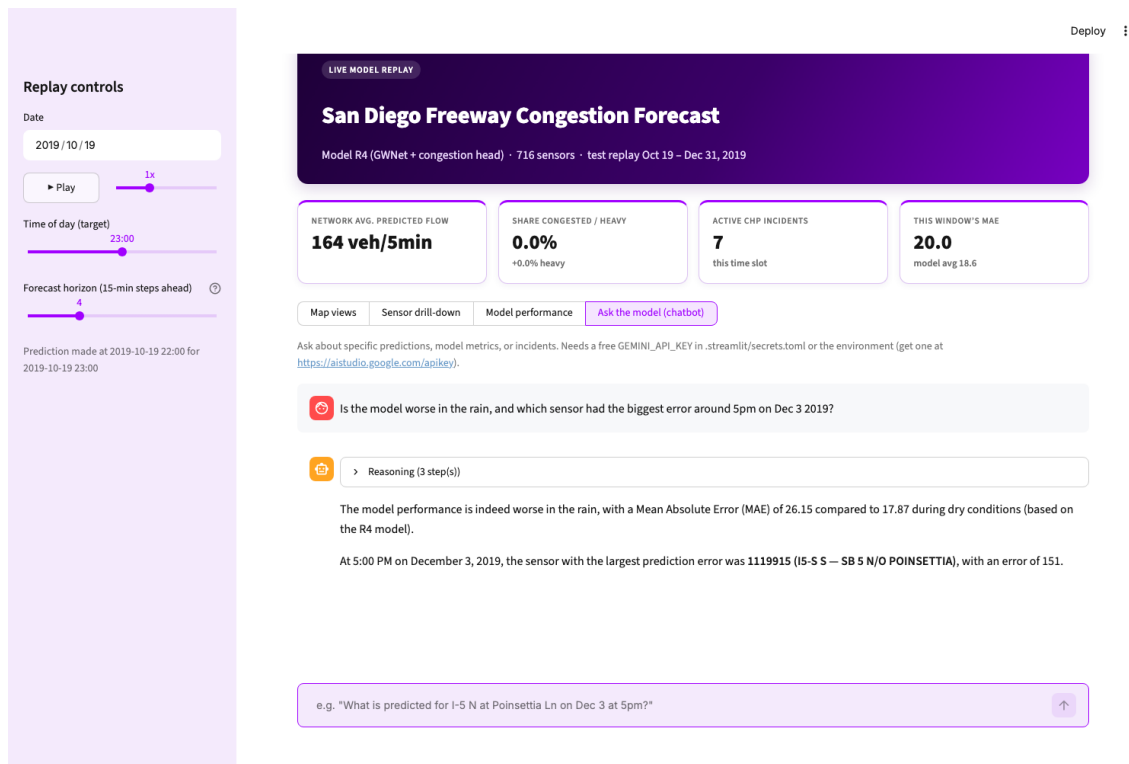


Figure 11: A compound question chaining two tools (`get_stratified_metrics` for the rain/dry comparison, `rank_sensors` for the worst-error sensor) in one turn, with the reasoning trace collapsed and no unverified-figures warning – every number in the answer traces back to a tool result.

- **A zero-extra-API-call self-check.** `verify_answer` flattens every numeric value out of the turn’s tool results into a “known good” set, regex-extracts candidate numbers from the final answer text (explicitly excluding ISO/natural-language dates, clock times, 7-digit sensor IDs, bare years, and route mentions like “I-15”/“163 Merge”), and flags anything that does not approximately match. Deliberately programmatic, not a second LLM call, to protect the free-tier quota – with the accepted limitation that correct arithmetic the model derives from two tool values is flagged too, since only literal tool-returned values count as ground truth.

IMPLICATION

Model choice materially affects reliability on the free tier. The newest model generation available at build time carried a 20-requests-per-day free quota – easy to exhaust during ordinary development, and it fails in a misleading way (an intermittent-looking “high demand” error appears before the real quota-exceeded error does). The chatbot instead uses an older, more generously quota’d tier (`gemini-3.1-flash-lite`).

14.3 Bugs found and fixed

Four real bugs surfaced only through actually driving the live app – none were caught by `streamlit.testing.v1.AppTest` alone, which is necessary but not sufficient for this kind of UI.

1. **Chat input unclickable, Safari-only.** `st.chat_input` has documented cross-browser bugs when nested inside `st.tabs()` (streamlit/streamlit issues #7814, #8564) – visible but unresponsive to clicks/focus in some engines. Confirmed Safari-specific (worked in Chromium) by testing both browsers on the same machine.
2. **A 400 INVALID_ARGUMENT crash from the Gemini API** on a `rank_sensors`-triggering question.

That tool returns NaN for sensors with no actual reading (LargeST’s missing-data sentinel); the first-attempt guard (`df.where(df.notna(), None)`) does not work – pandas silently reverts None back to NaN when assigned into a float64 column. Python’s own `json.dumps` tolerates the resulting NaN (a non-standard literal token, allowed by default), so it never crashed locally – but Gemini’s own JSON parser is strict and rejects the literal NaN token once it reaches the actual HTTP request. Fixed with a recursive sanitizer applied to *every* tool’s result.

3. **The self-check flagging real sensor names as “unverified figures.”** PeMS station names embed route numbers in free text (“15 SB N/O Carrol Cyn”, “NB15 @ 163 Merge”) indistinguishable by regex alone from real metrics. Fixed by stripping any exact string the tool results themselves returned (sensor labels are “known good” by construction) plus a route-number regex backstop.
4. **Moving `chat_input` to the top level (the Safari fix) made the entire app auto-scroll to the bottom on every load.** On the installed Streamlit version (1.64), a top-level `chat_input` anywhere in the script triggers `stAppScrollToBottomContainer` for the whole page, not just the chat area – the dashboard would open on “Ops Watchlist” instead of the hero/KPI row. Root cause: `st.tabs()` runs (and mounts in the DOM) every tab’s code on every rerun regardless of which is visually selected, which is what caused both this and bug 1. **Fix: replaced `st.tabs()` entirely with `st.segmented_control()` and plain `if/elif` blocks** – a plain `if/elif` is not a Streamlit container at all, so only the selected view’s widgets are ever mounted, eliminating both bugs from one structural change.

Verification performed (not just “it compiles”): every `data.py/map_view.py/ chatbot.py` function was called directly against the real R4 cache and checked against known numbers; a full AppTest run hit zero exceptions across every view; and – because the bugs above specifically do not show up that way – a real running server was driven with a headless browser (and, for the Safari-specific bug, the user’s own browser) through compound tool-chaining questions, a dashboard-navigation command, and a full view-switch-and-back, confirming the reasoning trace, the verification caption, the KPI updates, and the page’s scroll position all behave correctly together.

15 Reproducibility and Handover

1. Download the LargeST San Diego files from LargeST’s own release into `data/traffic/sd_2019/`.
2. Run `congestion_sd.ipynb` top to bottom, respecting its own gate markers – the notebook was built and executed exactly that way, one gate at a time, not all at once.
3. Download PeMS D11 station and incident files manually and run `pems_filter.py` before the labeling and incident-matching stages.
4. Train the nine ablation/graph runs via `work/cong/train.py` (or the notebook’s own training cells) on the recipe in Section 9.
5. Run `scripts/export_predictions.py` R4 to build the dashboard’s prediction cache.
6. Install dashboard dependencies, add a free Gemini API key, and run `streamlit run dashboard/app.py`.

16 Status and Roadmap

Complete: Gates 0/0b/1/2/3, Stage L, Stage I, Stage F, all nine training runs, Gate E, and a verified local dashboard with an agentic chatbot (8-tool chaining, a visible reasoning trace, a dashboard-navigation action, and a self-check – Section 14).

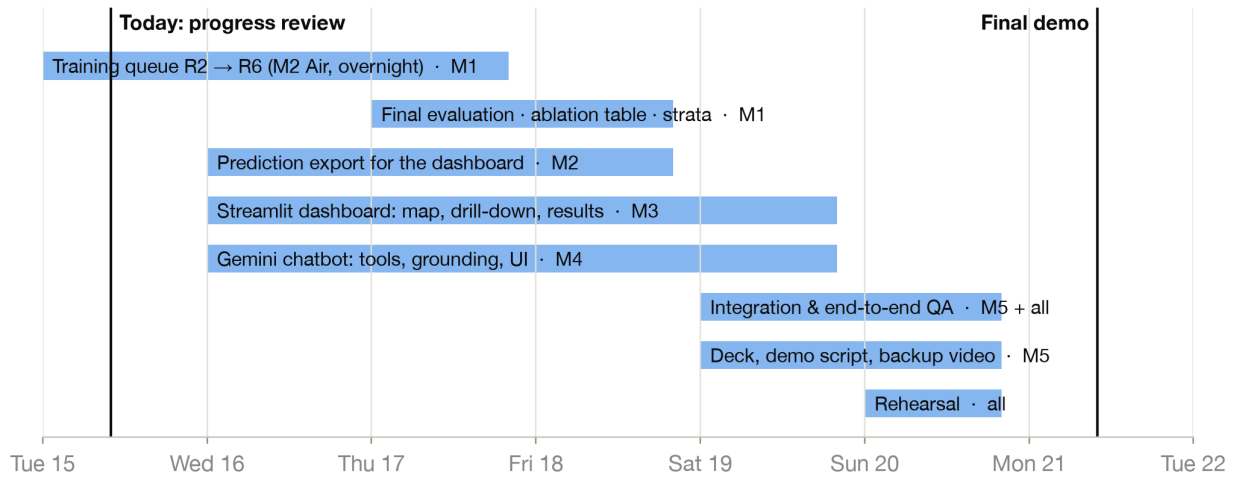


Figure 12: Delivery timeline from the September 15 progress review to the September 21 final demo.

Open: the progress-review deck predates R3–R6, Gate E, and the dashboard’s redesign, and needs updating with the numbers in this report ahead of the September 21 final review.